ASE CHAPTER 4

T3 Compare & Analyze Exercises Pack · Software Project Planning

Section 1: Comparative Analysis Frameworks (5 Items)

Exercise 4.3. Semicolon Counting vs. Lines of Code (LOC) in C Programs

A proposal is made to count the size of 'C' programs by number of semicolons, except those occurring with literal strings. Discuss the strengths and weaknesses to this size measure when compared with the lines of code count.

Comparative Analysis: Semicolon Count vs. Lines of Code (LOC): In the 'C' programming language, statements are syntactically terminated by semicolons (;), making semicolon counting a proxy for logical source statements. When compared with traditional physical Lines of Code (LOC), this approach demonstrates distinct trade-offs:

Strengths of Semicolon Counting:

- **1. Formatting Style Immunity:** Physical LOC is heavily distorted by indentation styles, brace placement (e.g., K&R vs. Allman styles, where { occupies a standalone line), and blank lines. Semicolon counting is completely invariant to whitespace, indentation, and newlines.
- **2. Multi-Statement Line Detection:** A developer can pack three statements onto a single physical line (`x = 1; y = 2; z = 3;`). Physical LOC counts this as 1 line, whereas semicolon counting correctly detects 3 distinct logical instructions.
- **3. Automation with language-aware tooling:** A lexer or parser can count semicolon-delimited statements in C while recognizing comments and string literals. A simple character scan is insufficient, and the count still reflects this chosen convention rather than a universal LOC standard.

Weaknesses and Limitations of Semicolon Counting:

- **1. Under-Counting Complex Control Constructs:** Compound statements and control structures in C do not terminate with semicolons: `if (...) { ... }`, while `(...) { ... }`, and for `(...) { ... }`. Relying on semicolons ignores the structural volume of decision blocks.
- **2. Header and Preprocessor Exclusion:** Critical architectural code in C—such as #define, #include, #ifdef, and macro templates—does not use semicolons. A program relying extensively on macros would appear artificially tiny.
- **3. The for-Loop Anomaly:** A single for loop statement inherently contains two semicolons (for `(i=0; i<n; i++)`), causing a single loop construct to be counted as 2 units, which distorts functional parity.
- **4. Language Dependency Persists:** Counting semicolon-terminated statements is a convention tied to languages that use semicolons that way; it is not a universal LOC rule and should not be

transferred unchanged to other languages or notations.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.1, pp. 143–145; Companion Slides 7–9.

Exercise 4.7. Function Points vs. Lines of Code: Multi-Dimensional Comparison

What are the size metrics? How is function point metric advantageous over LOC metric? Explain.

Software size can be measured using Lines of Code (LOC) or Function Points (FP).

- **LOC** counts source-code lines or statements. Its count depends on the programming language and reflects how the system is implemented.
- **Function Points** estimate user-visible functionality by counting weighted inputs, outputs, inquiries, internal logical files, and external interface files. They are linked to requirements and can be estimated before coding.

Compared with LOC, Function Points are less dependent on the implementation language and are easier for non-technical users to relate to system functionality. Changes in requirements can also be reflected through a revised function-point estimate.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.1–§4.3.2, pp. 143–150; Companion Slides 6, 10, 31.

Exercise 4.12. Comparative Analysis: Walston-Felix Model vs. NASA SEL Model

Explain the Walston & Felix model and compare with the SEL model.

Both are static, single-variable models that estimate effort and schedule from size L in KLOC.

- **Walston–Felix:** $E = 5.2L^{0.91}$ person-months and $D = 4.1L^{0.36}$ months.
- **SEL:** $E = 1.4L^{0.93}$ person-months and $D = 4.6L^{0.26}$ months.

For the same L, the equations can produce different estimates because their fitted coefficients and exponents differ. For example, at $L = 100$ KLOC, Walston–Felix gives about 344 PM and SEL about 101 PM. This comparison alone does not establish which estimate is more accurate for a particular project; that depends on the model's applicability and supporting project data.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.4, pp. 150–152; Companion Slides 34–40.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- المعادلتان تعطيان نتائج مختلفة للحجم نفسه؛ لا تختَر إحداهما على أنها أدق من دون التحقق من ملاءمتها وبيانات المشروع.

Synthesis Q3.1. Comparative Analysis of COCOMO '81 Operating Modes

Compare and contrast the three operating modes of COCOMO '81 (Organic, Semidetached, and Embedded) across project characteristics, team experience, environmental constraints, and governing mathematical coefficients.

COCOMO '81 Operating Modes Comparative Framework: Barry Boehm categorized software projects into three operating modes reflecting organizational complexity, constraint rigidity, and environmental familiarity (§4.4.1, pp. 152–155):

1. Organic Mode:

- **Size Range:** Typically small, ≤ 50 KLOC.
- **Team Experience:** Experienced, cohesive in-house team with deep familiarity with the organizational domain and existing systems.
- **Environmental Constraints:** Relaxed, flexible interface specifications; minimal hardware concurrency constraints; changes can be negotiated easily with users.
- **Basic Equations:** Effort $E = 2.4(\text{KLOC})^{1.05}$ PM, Schedule $D = 2.5(E)^{0.38}$ Months.
- **Intermediate Coefficients:** $a_i = 3.2$, $b_i = 1.05$.

2. Semidetached Mode:

- **Size Range:** Medium, typically $50 - 300$ KLOC.
- **Team Experience:** Mixed team experience; some members possess domain expertise, while others are new to the platform or application architecture.
- **Environmental Constraints:** Medium rigidity; some interfaces and operational protocols are locked, while internal implementation details remain flexible.
- **Basic Equations:** Effort $E = 3.0(\text{KLOC})^{1.12}$ PM, Schedule $D = 2.5(E)^{0.35}$ Months.
- **Intermediate Coefficients:** $a_i = 3.0$, $b_i = 1.12$.

3. Embedded Mode:

- **Size Range:** Typically large, > 300 KLOC (or any size operating under rigid operational constraints).
- **Team Experience:** Highly disciplined team operating under tight regulatory, verification, and audit standards.
- **Environmental Constraints:** Extreme rigidity; software must execute within unforgiving hardware limitations, strict real-time deadlines, and tightly coupled external interfaces (e.g., flight avionics, medical implants).
- **Basic Equations:** Effort $E = 3.6(\text{KLOC})^{1.20}$ PM, Schedule $D = 2.5(E)^{0.32}$ Months.
- **Intermediate Coefficients:** $a_i = 2.8$, $b_i = 1.20$.

Critical Analysis of the Exponent (b): The effort exponent grows progressively: **Organic** (1.05) $\rightarrow$ **Semidetached** (1.12) $\rightarrow$ **Embedded** (1.20). Since each effort exponent exceeds 1.0, the model estimates effort rising faster than size within each mode. This is a model-implied diseconomy of scale; the equations alone do not establish its cause.

> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.4.1–§4.4.2, pp. 152–158; Companion Slides 41–46.*

Synthesis Q3.2. Architectural Evolution: COCOMO '81 vs. COCOMO II

Analyze the architectural evolution from COCOMO '81 to COCOMO II, highlighting how the model was adapted to modern software engineering practices.

Evolutionary Drivers: Why COCOMO '81 Required Modernization: COCOMO '81 was constructed around the classical Waterfall lifecycle, centralized mainframe architectures, and custom code written from scratch in third-generation procedural languages (C, Fortran, Pascal). During the 1990s, software engineering shifted toward object-oriented design, commercial-off-the-shelf (COTS) integration, GUI builders, code generators, and iterative prototyping. COCOMO II was developed to address these fundamental changes:

Key Architectural Contrasts:

- **1. Lifecycle Coverage and Estimation Stages:**
- *COCOMO '81:* A monolithic model applied across the entire development lifecycle, requiring lines-of-code estimates even before requirements were stable.
- *COCOMO II:* Formulates **three progressive lifecycle stages**: Application Composition (prototyping/UI), Early Design (high-level architecture), and Post-Architecture (full development).
- **2. Sizing Metric Flexibility:**
- *COCOMO '81:* Exclusively sized using Delivered Source Instructions / Lines of Code (KDSI/KLOC).
- *COCOMO II:* Multi-metric sizing: **Object Points (NOP)** for Stage 1, **Unadjusted Function Points (UFP)** for Stage 2, and **Source Lines of Code (SLOC)** with formal reuse/breakage formulas for Stage 3.
- **3. Replacement of Rigid Modes with Dynamic Scale Factors:**
- *COCOMO '81:* Locked projects into three fixed, subjective modes (Organic, Semidetached, Embedded), resulting in sharp, discontinuous jumps in effort estimates.
- *COCOMO II:* Replaced discrete modes with **5 Scale Factors (SF_j)**: Precedentedness (PREC), Development Flexibility (FLEX), Architecture/Risk Resolution (RESL), Team Cohesion (TEAM), and Process Maturity (PMAT). The exponent is $B = 0.91 + 0.01 \sum SF_j$; the chapter's scale-factor values give a range of 0.91 to 1.23.
- **4. Refined Cost Driver Multipliers:**
- *COCOMO '81:* Utilized 15 static cost drivers calibrated to 1980s mainframe development environments.
- *COCOMO II:* Expanded to **17 Post-Architecture drivers** reflecting modern realities: software reuse (RUSE), platform volatility (PVOL), multi-site development (SITE), and personnel turnover (PCON).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.5, pp. 161–171; Companion Slides 73–86. [Curriculum Invariant]

Section 2: Analytical, Justification & Evaluative Exercises (11 Items)

Exercise 4.8. Feasibility of Pre-Coding Software Size Estimation

Is it possible to estimate software size before coding? Justify your answer with a suitable example.

Feasibility Assertion: Yes, software size can and must be estimated prior to coding. In professional software engineering, initial size estimation is the mandatory baseline required to establish project budgets, schedule staffing, and contractually commit to delivery deadlines before investment capital is allocated.

Technical Justification Mechanisms:

- **1. Function Point Analysis from the SRS:** Software requirements describe functional data flows regardless of whether code exists. By auditing the Software Requirements Specification (SRS), systems analysts can count:
- Every user input screen and data entry transaction $\rightarrow$ **External Inputs (EI)**.
- Every generated management report, graph, and billing document $\rightarrow$ **External Outputs (EO)**.
- Every interactive lookup and search interface $\rightarrow$ **External Inquiries (EQ)**.
- Every persistent database table maintained inside the system $\rightarrow$ **Internal Logical Files (ILF)**.
- Every shared interface or foreign database connection $\rightarrow$ **External Interface Files (EIF)**.
- **2. Work Breakdown Structure (WBS) Decomposition:** The system is decomposed hierarchically into subsystems, modules, and sub-routines. Experienced architects estimate individual module sizes using three-point PERT statistical estimation ($S = \frac{S_{\text{opt}} + 4S_m + S_{\text{pess}}}{6}$).
- **3. Analogy-Based Sizing:** Comparing the proposed system to previously completed, historical projects with similar architectural patterns (e.g., estimating an e-commerce platform by comparing it to an earlier portal built by the same organization).

Concrete Example: In an automated banking system, the requirements mandate: - Customer Account Registration (1 EI) - Funds Transfer Transaction (1 EI) - Account Balance Statement Generation (1 EO) - Transaction History Inquiry (1 EQ) - Customer Account Database Store (1 ILF) - External Inter-Bank Clearing Network Interface (1 EIF)

Using Albrecht's Average weights:

$$\text{UFP} = (2 \times 4) + (1 \times 5) + (1 \times 4) + (1 \times 10) + (1 \times 7) = 8 + 5 + 4 + 10 + 7 = 34 \text{ UFP}$$

Assuming a nominal CAF of 1.00, the system size is objectively baselined at 34 FP before a single line of Java or SQL code has been written.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.2, pp. 145–148; Companion Slides 10–25.

Exercise 4.17. Basic COCOMO Productivity Relation & Diseconomy of Scale

Using the basic COCOMO model, under all three operating modes, determine the performance relation for the ratio of delivered source code lines per person-month of effort. Determine the reasonableness of this relation for several types of software projects.

Mathematical Derivation of the Productivity Relation: In Basic COCOMO, development effort is defined as $E = a_b(\mathrm{KLOC})^{b_b}\ \mathrm{PM}$, where size L in lines of code is $L = 1000 \times \mathrm{KLOC}$. Productivity ($P$), defined as delivered source lines per person-month ($P = \frac{L}{E}$), is derived as:

$$P = \frac{1000 \times \mathrm{KLOC}}{a_b(\mathrm{KLOC})^{b_b}} = \frac{1000}{a_b} \times (\mathrm{KLOC})^{1-b_b}\ \mathrm{LOC/PM}$$

Evaluation Across the Three Operating Modes:

- **1. Organic Mode ($a_b = 2.4, b_b = 1.05$):**

$$P_{\text{organic}} = \frac{1000}{2.4} \times (\mathrm{KLOC})^{1-1.05} = 416.67 \times (\mathrm{KLOC})^{-0.05}\ \mathrm{LOC/PM}$$

- **2. Semidetached Mode ($a_b = 3.0, b_b = 1.12$):**

$$P_{\text{semidetached}} = \frac{1000}{3.0} \times (\mathrm{KLOC})^{1-1.12} = 333.33 \times (\mathrm{KLOC})^{-0.12}\ \mathrm{LOC/PM}$$

- **3. Embedded Mode ($a_b = 3.6, b_b = 1.20$):**

$$P_{\text{embedded}} = \frac{1000}{3.6} \times (\mathrm{KLOC})^{1-1.20} = 277.78 \times (\mathrm{KLOC})^{-0.20}\ \mathrm{LOC/PM}$$

Analysis of Reasonableness (Diseconomy of Scale):

- **Model-implied size relation:** In all three modes, $b_b > 1$, so the exponent $(1 - b_b)$ is negative (-0.05, -0.12, and -0.20). With the mode coefficients held fixed, the model's estimated project productivity in LOC/PM therefore decreases as KLOC increases.
- This is a relationship implied by the equations; it does not by itself measure an individual programmer's productivity or establish why productivity changes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.4.1, pp. 152–155; Companion Slides 44–47.

Exercise 4.18. Phase-Wise Effort Sensitivity to Cost Drivers

The effort distribution for a 240 KLOC organic mode software development project is: product design 12%, detailed design 24%, code and unit test 36%, integrate and test 28%. How would the following changes, from low to high, affect the phase distribution of effort and the total effort: analyst capability, use of modern programming languages, required reliability, requirements volatility?

Total-effort effect from the COCOMO ’81 multipliers: Holding size and the other drivers fixed, moving one driver from Low to High changes estimated total effort by the ratio of its High multiplier to its Low multiplier:

- **Analyst capability (ACAP):** $0.71/1.46 \approx 0.486$; estimated total effort falls by about 51.4%.
- **Modern programming practices (MODP):** $0.82/1.24 \approx 0.661$; estimated total effort falls by about 33.9%.
- **Required reliability (RELY):** $1.40/0.75 \approx 1.867$; estimated total effort rises by about 86.7%.

Phase-distribution limit: Detailed COCOMO applies phase-sensitive multipliers. The baseline percentages alone do not give revised phase shares; those require the corresponding phase-level multipliers. Do not infer exact new percentages from the total-effort multipliers.

Requirements volatility: COCOMO ’81 has no separate requirements-volatility driver among its 15 cost drivers. It can motivate a qualitative rework discussion, but the model does not provide a direct Low-to-High multiplier for it.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.4.3, pp. 158–160; Companion Slides 58–63.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- كل محرك كلفة يؤثر على مرحلة محددة بقوة أكبر:
- كفاءة المحلل (ACAP) ← تؤثر بقوة على مرحلة التصميم المبدئي (Product Design).
- لغات البرمجة الحديثة (MODP) ← تضغط مرحلة كتابة الكود وفحص الوحدات (Coding).
- الموثوقية المطلوبة (RELY) ← تضخم مرحلة التكامل والفحص (Test & Integration).

Exercise 4.24. COCOMO II Stages and the Limits of Popularity Claims

Describe various stages of COCOMO-II. Which stage is more popular and why?

The Three Stages of COCOMO II: COCOMO II provides three estimation stages matched to the information available during development:

- **1. Application Composition:** Uses Object Points for screens, reports, and 3GL components; suited to application composition and prototyping.
- **2. Early Design:** Uses Function Points or KLOC with five scale factors and seven aggregated effort multipliers when detailed architecture and driver information are not yet available.
- **3. Post-Architecture:** Uses SLOC or Function Points with five scale factors and 17 detailed cost drivers after the architecture is established.

Stage choice: Post-Architecture supports a more realistic and detailed estimate when its inputs are available. Early Design is appropriate earlier, when fewer project details are known. The course material supports this information-maturity distinction; it does not give industry adoption statistics.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.5, pp. 161–171; Companion Slides 73–86.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- Post-Architecture يعطي تقديراً أكثر تفصيلاً بعد توفر المعمارية ومدخلات المحركات السبعة عشر. لكن المادة لا تقدم بيانات عن مدى انتشار كل مرحلة؛ لذلك لا يمكن الجزم أيها «الأكثر شعبية» أو لماذا.

Exercise 4.27. The Trade-Off Between Time and Cost in Putnam's Model

Describe the trade-off between time versus cost in Putnam resource allocation model.

Mathematical Foundation of the Time-Cost Trade-Off: Putnam's Software Equation relates product size (S), total development effort (K), and development time (t_d):

$$S = C_k \cdot K^{1/3} \cdot t_d^{4/3}$$

Solving for effort (K) as a function of schedule (t_d):

$$K = \frac{S^3}{C_k^3 \cdot t_d^4} \implies K \propto \frac{1}{t_d^4}$$

Engineering Implications of the 4th Power Law:

- **1. Fourth-power relation:** With product size and technology factor fixed, effort is inversely proportional to t_d^4.
- For example, compressing schedule by just **15%** ($t'_d = 0.85t_d$) increases required effort by $\frac{1}{(0.85)^4} \approx 1.92$—**nearly doubling the project cost**.
- Compressing schedule by **50%** ($t'_d = 0.5t_d$) multiplies the modelled effort by $2^4 = 16$; this is 16 times the baseline, or a 1500% increase.
- **2. Schedule extension:** Extending development time by 20% changes effort by $1/(1.20)^4 \approx 0.482$, a reduction of about 51.8%, under the same fixed-size/fixed-technology assumptions.
- The equation describes the model's effort-time trade-off. It does not by itself establish a universal minimum feasible schedule or an “impossible region.”

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.6.4, printed pp. 187–188; Companion Slides 149–150, 166–173.

Exercise 4.28. Derivation of the Putnam Software Equation

Discuss the Putnam resources allocation model. Derive the time and effort equations.

Derivation of the Putnam Software Equation: The derivation begins with Peter Norden's Rayleigh manpower utilization equation (§4.6.1, printed pp. 182–185):

$$m(t) = 2Kat \cdot e^{-at^2}$$

Step 1: Establishing the Shape Parameter (a): The peak of the Rayleigh curve occurs when $\frac{dm}{dt} = 0$:

$$\frac{dm}{dt} = 2Kae^{-at^2}(1 - 2at^2) = 0 \implies 1 - 2at^2 = 0 \implies a = \frac{1}{2t_d^2}$$

Substituting a back into $m(t)$ gives:

$$m(t) = \frac{K}{t_d^2} \cdot t \cdot e^{-\frac{t^2}{2t_d^2}}$$

Step 2: Difficulty and Manpower Build-Up Metrics: Putnam distinguishes the difficulty metric D from the manpower build-up metric D_0:

$$D = \frac{K}{t_d^2} \quad \text{and} \quad D_0 = \frac{K}{t_d^3}$$

The initial slope of the Rayleigh staffing curve is $m\prime(0) = K/t_d^2 = D$; it is not D_0.

Step 3: Empirical Linking of Size to Difficulty and Effort: Through regression analysis of historical systems, Putnam observed that software size (S in source lines of code) is proportional to the product of total effort and the manpower acceleration metric:

$$S = C_k \cdot K^{1/3} \cdot t_d^{4/3}$$

Step 4: Solving for Governing Effort and Schedule Equations: - **Effort Equation:** Cubing both sides:

$$S^3 = C_k^3 \cdot K \cdot t_d^4 \implies K = \frac{S^3}{C_k^3 \cdot t_d^4}$$

- **Schedule Equation:** Isolating t_d:

$$t_d^4 = \frac{S^3}{C_k^3 \cdot K} \implies t_d = \left(\frac{S^3}{C_k^3 \cdot K}\right)^{1/4} = \left(\frac{S}{C_k \cdot K^{1/3}}\right)^{3/4}$$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §§4.6.1–4.6.4, printed pp. 182–188; Companion Slides 125–130, 149–150, 164–173.

Exercise 4.31. Estimating Before Detailed Requirements & Design: The Planning Paradox

It seems odd that cost and size estimates are developed during software project planning—before detailed software requirements analysis or design has been conducted. Why do we think this is done? Are there circumstances when it should not be done?

Why Pre-Requirements Estimation is Conducted (Business Justification): Early estimates can support planning decisions before detailed requirements and design are complete, but they depend on limited information and should be treated as provisional and revised as the project becomes clearer.

- **1. Planning and feasibility:** An approximate size, effort, and cost help decision-makers assess whether a proposed project is worth pursuing.
- **2. Budgeting and resource planning:** Early estimates help plan funding, staffing, and schedule while details are still being refined.
- **3. Bidding or authorization:** A proposal may require an estimate before detailed design is available; its assumptions and uncertainty should be stated.

When a fixed, precise estimate is not defensible:

- When requirements, scope, or technical feasibility are too uncertain to support a meaningful point estimate, state assumptions and a range or defer commitment until more evidence is available.
- As information changes, revise the estimate; an early planning estimate is not a guarantee of final cost or duration.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.1, pp. 139–141; Companion Slides 4–5.

Exercise 4.32. Analysis of Typical Software Risks & Staff Turnover

Discuss typical software risks. How does the staff turnover problem affect software projects?

The five typical software-risk categories identified in the chapter are:

- **Dependencies on outside agencies or factors:** Availability of trained, experienced people or customer-furnished information.
- **Requirement issues:** Unclear, unprioritized, or changing requirements.
- **Management issues:** Inadequate planning, unrealistic expectations, or poor communication.
- **Lack of knowledge:** Inadequate training or insufficient domain experience.
- **Other risk categories:** Turnover of essential personnel, unavailable testing facilities, or unachievable performance requirements.

Impact of the Staff Turnover Problem: Staff turnover can leave a project without people who hold needed experience, interrupting continuity and requiring the team to restore that capability. The chapter lists turnover of essential personnel as a typical risk; it does not quantify a universal schedule or cost effect.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.7.2, pp. 195–196; Companion Slides 184–189.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- دوران الكادر الأساسي مذكور ضمن المخاطر الشائعة. أثره يتوقف على المشروع؛ لا تفترض مقداراً ثابتاً للتأخير أو الكلفة من دون بيانات.

Exercise 4.35. Economic Justification of Risk Management

What is risk? Is it economical to do risk management? What is the effect of this activity on the overall cost of the project?

> **Definition of Risk:** A risk is a problem that may cause loss or threaten project success but has not happened yet.
>
> **Economic Effect:** Risk management adds planning and control work to a project. It is economically justified when that cost is lower than the expected loss it helps avoid or reduce. Risk Exposure supports this comparison because it combines the probability of loss with its potential magnitude. The course source provides no universal ROI, budget percentage, or fixed savings figure.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.7, pp. 194–198; Companion Slides 180–183.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - لا يحدد المصدر نسبة ثابتة من الميزانية أو مقدار توفير عام؛ قارن كلفة الإجراء بالخسارة المتوقعة التي قد يخفضها.

Exercise 4.36. Risk Analysis of an Academic Student Software Project

There are significant risks even in student projects. Analyze a student project and list all the risks.

A student project can face risks from each of the chapter's five categories:

- **Dependencies on outside agencies or factors:** delayed access to a dataset, service, or required decision.
- **Requirement issues:** unclear acceptance criteria or changing requested features.
- **Management issues:** underestimated work or weak coordination.
- **Lack of knowledge:** unfamiliar tools or insufficient domain knowledge.
- **Other risks:** loss of a key contributor, unavailable test facilities, or a performance target that proves difficult to meet.

These are illustrative examples for a student project; the categories are the chapter's taxonomy.

Source for the five categories: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.7.2, pp. 195–196; Companion Slides 184–189.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- الأمثلة تختلف من مشروع لآخر؛ صنّف كل خطر ضمن إحدى الفئات الخمس، ولا تعتبر الأمثلة قائمة حصرية.

Synthesis Q3.3. Check the Printed Phase-Allocation Totals

For the Organic Small row in the phase-allocation table, list the printed effort and schedule fractions, calculate each total, and state how the figures should be reported.

Printed values for Organic Small ($S \approx 2$): In Book Table 4.7 (printed p. 159) and companion Slides 62–63, the five phases are Plan & Requirement, System Design, Detail Design, Module Code & Test, and Integration & Test.

- **Effort (μ_p), in that phase order:** $(0.06, 0.16, 0.26, 0.42, 0.16)$; total $0.06 + 0.16 + 0.26 + 0.42 + 0.16 = 1.06$ (**106%**).
- **Schedule (τ_p), in that phase order:** $(0.10, 0.19, 0.24, 0.39, 0.18)$; total $0.10 + 0.19 + 0.24 + 0.39 + 0.18 = 1.10$ (**110%**).

Treatment: Report the source values and their arithmetic totals as printed. The source does not explain why the totals exceed 100%. If a problem requires allocating a fixed project total across phases, state whether the raw fractions are used directly or normalized; do not silently present normalized values as the source table.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 159, Table 4.7; Companion Slides 62–63.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- قيم Organic Small المطبوعة تجمع إلى 106% للجهد و110% للجدول الزمني. لا يشرح المصدر سبب الزيادة؛ إذا طُلب توزيع إجمالي ثابت، وضّح هل تستخدم القيم الخام أو تعيد تطبيعها.

Topic-to-Question Map

Topic Area	Questions Covering Item	Primary Course Sources	Key Comparative & Analytical Concepts
1. Size Estimation (LOC & Function Count)	Ex 4.3, Ex 4.7, Ex 4.8	K.K. Aggarwal pp. 143–150; Slides 7–32	Semicolons vs LOC in C, FPA vs LOC multi-dimensional comparison, Pre-coding size estimation feasibility.
2. Cost Estimation Models (Static)	Ex 4.12, Ex 4.31	K.K. Aggarwal pp. 139–141, 150–152; Slides 4–5, 34–40	Walston-Felix vs NASA SEL empirical models, Pre-requirements estimation paradox & business justification.
3. COCOMO '81 (Basic, Interm., Detailed)	Synthesis Q3.1, Ex 4.17, Ex 4.18, Synthesis Q3.3	K.K. Aggarwal pp. 152–160; Slides 41–65	3 modes comparison, Productivity relation $\text{Prod} = \frac{1000}{a_b}\text{KLOC}^{1-b_b}$, Phase sensitivity to drivers, Book Table 4.7 106% defect.
4. COCOMO II (Evolution & Stages)	Synthesis Q3.2, Ex 4.24	K.K. Aggarwal pp. 161–171; Slides 73–86	COCOMO '81 vs II architectural evolution, Comparative evaluation of 3 stages & Post-Architecture popularity.
5. Putnam Resource Allocation Model	Ex 4.27, Ex 4.28	K.K. Aggarwal printed pp. 182–188; Slides 123–140, 164–173	Schedule compression 4th-power law ($K \propto t_d^{-4}$), Derivation of software equation $S = C_k K^{1/3} t_d^{4/3}$.
6. Software Risk Management	Ex 4.32, Ex 4.35, Ex 4.36	K.K. Aggarwal pp. 194–198; Slides 180–193	Typical software risks & staff turnover analysis, Economic ROI of risk management, Student project risk profile.